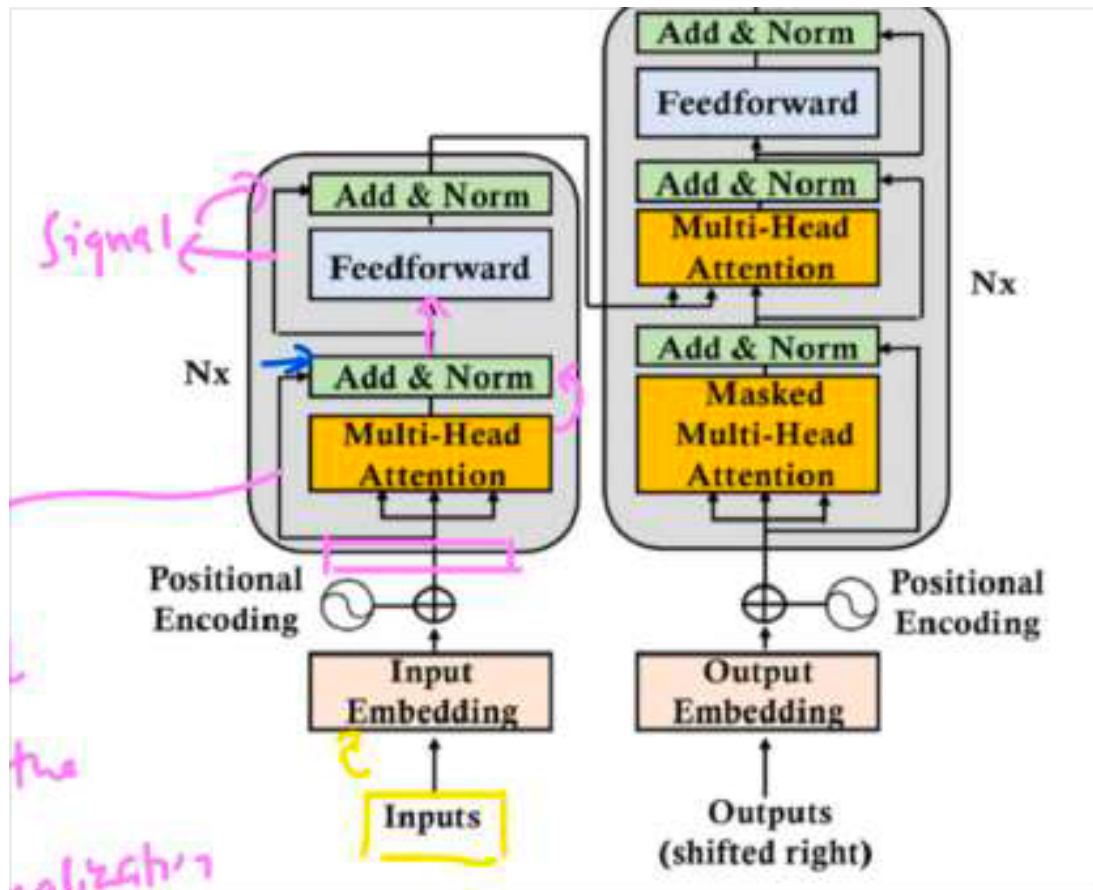


# Encoder-Decoder full walkthrough



## ✅ Step 1 Recap — Encoder Input

We started with two words: "CAT" and "The".

Each word goes through:

- **Embedding**
- **Positional Encoding**
- Then summed → Final Input to Encoder

| Word | Embedding            | Positional Encoding  | Final Input to Encoder |
|------|----------------------|----------------------|------------------------|
| CAT  | [0.5, 0.6, 0.7, 0.8] | [0.9, 0.1, 0.1, 0.9] | [1.4, 0.7, 0.8, 1.7]   |
| The  | [0.1, 0.2, 0.3, 0.4] | [0.1, 0.1, 0.1, 0.1] | [0.2, 0.3, 0.4, 0.5]   |

### Step 2 Recap — Encoder Output

We now have **context vectors** ( $z_1$ ,  $z_2$ ) which will be fed into the decoder.

| Word | Final Input to Encoder | Encoder Output ( $z$ ) |
|------|------------------------|------------------------|
| CAT  | [1.4, 0.7, 0.8, 1.7]   | [1.15, 1.09]           |
| The  | [0.2, 0.3, 0.4, 0.5]   | [0.35, 0.41]           |

These outputs are what the decoder will use to "look back" at the input via **encoder-decoder attention**.

### Step 3 Recap — Decoder Input Construction

We're feeding the decoder with the partial output sentence:

"I", "have"

Each word is converted into:

- A word embedding
- A positional encoding
- Their sum becomes the **final decoder input vector**

| Word | Embedding            | Positional Encoding      | Final Input to Decoder   |
|------|----------------------|--------------------------|--------------------------|
| I    | [0.1, 0.2, 0.3, 0.4] | [0.01, 0.02, 0.03, 0.04] | [0.11, 0.22, 0.33, 0.44] |
| have | [0.5, 0.6, 0.7, 0.8] | [0.05, 0.06, 0.07, 0.08] | [0.55, 0.66, 0.77, 0.88] |

## STEP 4: Masked Self-Attention in Decoder — Fully Manual Walkthrough

We already have these:

### Decoder Inputs (after embedding + position):

```
nginx
Token 0: "I"    →  $x_0 = [0.11, 0.22, 0.33, 0.44]$ 
Token 1: "have" →  $x_1 = [0.55, 0.66, 0.77, 0.88]$ 
```

Let's use these **projection matrices** to get Q, K, and V:

### Projection Matrices:

(These are 4D→2D)

```
ini
W_q =      W_k =      W_v =
[0.2 0.3]  [0.3 0.2]  [0.4 0.1]
[0.1 0.4]  [0.5 0.1]  [0.3 0.3]
[0.4 0.2]  [0.1 0.3]  [0.2 0.2]
[0.3 0.1]  [0.2 0.4]  [0.1 0.4]
```



Ask anything

+ Tools



#### ◆ Step 4.1: Compute Q for both tokens

$Q_0 = x_0 @ W_q$

markdown

Copy Edit

$x_0 = [0.11, 0.22, 0.33, 0.44]$

$Q_0[0] = 0.11 \times 0.2 + 0.22 \times 0.1 + 0.33 \times 0.4 + 0.44 \times 0.3$   
 $= 0.022 + 0.022 + 0.132 + 0.132 = 0.308$

$Q_0[1] = 0.11 \times 0.3 + 0.22 \times 0.4 + 0.33 \times 0.2 + 0.44 \times 0.1$   
 $= 0.033 + 0.088 + 0.066 + 0.044 = 0.231$

$\rightarrow Q_0 = [0.308, 0.231]$

$Q_1 = x_1 @ W_q$

markdown

Copy Edit

$x_1 = [0.55, 0.66, 0.77, 0.88]$

$Q_1[0] = 0.55 \times 0.2 + 0.66 \times 0.1 + 0.77 \times 0.4 + 0.88 \times 0.3$   
 $= 0.11 + 0.066 + 0.308 + 0.264 = 0.748$

$Q_1[1] = 0.55 \times 0.3 + 0.66 \times 0.4 + 0.77 \times 0.2 + 0.88 \times 0.1$   
 $= 0.165 + 0.264 + 0.154 + 0.088 = 0.671$

$\rightarrow Q_1 = [0.748, 0.671]$



#### ◆ Step 4.2: Compute $K_0$ and $K_1$

$K_0 = x_0 @ W_k$

mathematica

Copy Edit

$K_0[0] = 0.11 \times 0.3 + 0.22 \times 0.5 + 0.33 \times 0.1 + 0.44 \times 0.2$   
 $= 0.033 + 0.11 + 0.033 + 0.088 = 0.264$

$K_0[1] = 0.11 \times 0.2 + 0.22 \times 0.1 + 0.33 \times 0.3 + 0.44 \times 0.4$   
 $= 0.022 + 0.022 + 0.099 + 0.176 = 0.319$

$\rightarrow K_0 = [0.264, 0.319]$

$K_1 = x_1 @ W_k$

mathematica

Copy Edit

$K_1[0] = 0.55 \times 0.3 + 0.66 \times 0.5 + 0.77 \times 0.1 + 0.88 \times 0.2$   
 $= 0.165 + 0.33 + 0.077 + 0.176 = 0.748$

$K_1[1] = 0.55 \times 0.2 + 0.66 \times 0.1 + 0.77 \times 0.3 + 0.88 \times 0.4$   
 $= 0.11 + 0.066 + 0.231 + 0.352 = 0.759$

$\rightarrow K_1 = [0.748, 0.759]$



### ◆ Step 4.3: Compute Attention Scores

Using:

$$\text{Score}_{ij} = \frac{Q_i \cdot K_j^T}{\sqrt{d_k}} \text{ where } d_k = 2$$

Let's calculate the dot products:

$$\text{Score}[0][0] = Q_0 \cdot K_0 / \sqrt{2}$$

$$\begin{aligned} &= (0.308 \times 0.264 + 0.231 \times 0.319) / \sqrt{2} \\ &= (0.0813 + 0.0737) / 1.414 \\ &= 0.155 / 1.414 \approx 0.11 \end{aligned}$$

$$\text{Score}[0][1] = Q_0 \cdot K_1 / \sqrt{2}$$

$$\begin{aligned} &= (0.308 \times 0.748 + 0.231 \times 0.759) / \sqrt{2} \\ &= (0.230 + 0.175) / 1.414 \\ &= 0.405 / 1.414 \approx 0.286 \end{aligned}$$

But due to **masking**, token 0 can only attend to itself:

- So final `score[0] = [0.11, -∞]`

$$\text{Score}[1][0] = Q_1 \cdot K_0 / \sqrt{2}$$

$$\begin{aligned} &= (0.748 \times 0.264 + 0.671 \times 0.319) / 1.414 \\ &= (0.197 + 0.214) / 1.414 \\ &= 0.411 / 1.414 \approx 0.291 \end{aligned}$$

$$\text{Score}[1][1] = Q_1 \cdot K_1 / \sqrt{2}$$

$$\begin{aligned} &= (0.748 \times 0.748 + 0.671 \times 0.759) / 1.414 \\ &= (0.560 + 0.510) / 1.414 = 1.07 / 1.414 \approx 0.756 \end{aligned}$$

So:

- `score[1] = [0.291, 0.756]`

### ◆ Step 4.4: Apply Softmax

$$\text{Softmax}([0.11, -\infty]) = [1.0, 0.0]$$

$$\text{Softmax}([0.291, 0.756]):$$

$$\text{Shifted} = [0.291 - 0.756, 0.756 - 0.756] = [-0.465, 0]$$

makefile

$$\begin{aligned} \text{exp} &= [e^{-0.465} \approx 0.628, e^0 = 1.0] \\ \text{sum} &= 0.628 + 1.0 = 1.628 \end{aligned}$$

→ [0.385, 0.615]

Copy Edit

↑

#### ◆ Step 4.5: Compute $V_0, V_1$

$V_0 = x_0 @ W_v$

markdown

Copy Edit

$$\begin{aligned} V_0[0] &= 0.11 \times 0.4 + 0.22 \times 0.3 + 0.33 \times 0.2 + 0.44 \times 0.1 \\ &= 0.044 + 0.066 + 0.066 + 0.044 = 0.22 \end{aligned}$$

$$\begin{aligned} V_0[1] &= 0.11 \times 0.1 + 0.22 \times 0.3 + 0.33 \times 0.2 + 0.44 \times 0.4 \\ &= 0.011 + 0.066 + 0.066 + 0.176 = 0.319 \end{aligned}$$

$$\rightarrow V_0 = [0.22, 0.319]$$

$V_1 = x_1 @ W_v$

markdown

Copy Edit

$$\begin{aligned} V_1[0] &= 0.55 \times 0.4 + 0.66 \times 0.3 + 0.77 \times 0.2 + 0.88 \times 0.1 \\ &= 0.22 + 0.198 + 0.154 + 0.088 = 0.66 \end{aligned}$$

$$\begin{aligned} V_1[1] &= 0.55 \times 0.1 + 0.66 \times 0.3 + 0.77 \times 0.2 + 0.88 \times 0.4 \\ &= 0.055 + 0.198 + 0.154 + 0.352 = 0.759 \end{aligned}$$

↓

→  $V_1 = [0.66, 0.759]$

Copy Edit

↑

#### ◆ Step 4.6: Output = Attention Weight $\times V$

$$Z_0 = [1.0, 0.0] \times [V_0, V_1] = V_0 = [0.22, 0.319]$$

$$Z_1 = [0.385, 0.615] \times [V_0, V_1]$$

Copy Edit

$$Z_1[0] = 0.385 \times 0.22 + 0.615 \times 0.66 = 0.0847 + 0.4059 \approx 0.49$$

$$Z_1[1] = 0.385 \times 0.319 + 0.615 \times 0.759 = 0.1228 + 0.4669 \approx 0.59$$

$$\rightarrow Z_1 = [0.49, 0.59]$$

#### ✓ Final Output of Decoder's Self-Attention

| Token  | Z Output      |
|--------|---------------|
| "I"    | [0.22, 0.319] |
| "have" | [0.49, 0.59]  |

↓

## ✓ STEP 5: Encoder-Decoder Cross Attention (Full Calculation)

### 🔗 Purpose:

Each decoder token ("I", "have") now looks back at encoder outputs ( $z_1$ ,  $z_2$ ) using cross-attention.

### 💎 Input to this Step:

#### 📌 Decoder side (after self-attention):

```
arduino
Z0 = [0.22, 0.319] ← for "I"
Z1 = [0.49, 0.59] ← for "have"
```

These act as queries (Q).

#### 📌 Encoder outputs (fixed context vectors):

```
arduino
Z1 = [1.15, 1.09] ← for "CAT"
Z2 = [0.35, 0.41] ← for "The"
```

These are used as both keys (K) and values (V).



### 💎 Step 5.1: Define projection matrices

For simplicity, we'll define:

- $Q\_proj$  ( $2D \rightarrow 2D$ ) for decoder output  $\rightarrow$  query
- $K\_proj$  and  $V\_proj$  ( $2D \rightarrow 2D$ ) for encoder output  $\rightarrow$  key and value

Let's pick:

```
lua
W_Q = [[0.2, 0.1], [0.3, 0.4]]
W_K = [[0.5, 0.6], [0.7, 0.8]]
W_V = [[0.4, 0.5], [0.6, 0.7]]
```

### 💎 Step 5.2: Compute Q (decoder $Z \times W\_Q$ )

For  $Z_0 = [0.22, 0.319]$ :

```
Q0[0] = 0.22×0.2 + 0.319×0.3 = 0.044 + 0.0957 = 0.1397
Q0[1] = 0.22×0.1 + 0.319×0.4 = 0.022 + 0.1276 = 0.1496
→ Q0 = [0.1397, 0.1496]
```



For  $Z_1 = [0.49, 0.59]$ :

```
Q1[0] = 0.49×0.2 + 0.59×0.3 = 0.098 + 0.177 = 0.275
Q1[1] = 0.49×0.1 + 0.59×0.4 = 0.049 + 0.236 = 0.285
→ Q1 = [0.275, 0.285]
```

### ◆ Step 5.3: Compute K (encoder outputs × W\_K)

For  $z_1 = [1.15, 1.09]$ :

```
mathematica
K1[0] = 1.15×0.5 + 1.09×0.7 = 0.575 + 0.763 = 1.338
K1[1] = 1.15×0.6 + 1.09×0.8 = 0.69 + 0.872 = 1.562
→ K1 = [1.338, 1.562]
```

For  $z_2 = [0.35, 0.41]$ :

```
mathematica
K2[0] = 0.35×0.5 + 0.41×0.7 = 0.175 + 0.287 = 0.462
K2[1] = 0.35×0.6 + 0.41×0.8 = 0.21 + 0.328 = 0.538
```

→  $K_2 = [0.462, 0.538]$

### ◆ Step 5.4: Compute Attention Scores

Use:

$$\text{score}_{ij} = \frac{Q_i \cdot K_j^T}{\sqrt{2}}$$

For  $Q_0 = [0.1397, 0.1496]$ :

With  $K_1$ :

```
= (0.1397×1.338 + 0.1496×1.562) / 1.414
= (0.1868 + 0.2338) / 1.414 ≈ 0.4206 / 1.414 ≈ 0.297
```

With  $K_2$ :

```
= (0.1397×0.462 + 0.1496×0.538) / 1.414
= (0.0645 + 0.0805) / 1.414 = 0.145 / 1.414 ≈ 0.1025
```

→ So  $\text{score}_0 = [0.297, 0.1025]$

For  $Q_1 = [0.275, 0.285]$ :

With  $K_1$ :

$$\begin{aligned} &= (0.275 \times 1.338 + 0.285 \times 1.562) / 1.414 \\ &= (0.368 + 0.445) / 1.414 = 0.813 / 1.414 \approx 0.575 \end{aligned}$$

With  $K_2$ :

$$\begin{aligned} &= (0.275 \times 0.462 + 0.285 \times 0.538) / 1.414 \\ &= (0.127 + 0.153) / 1.414 = 0.280 / 1.414 \approx 0.198 \end{aligned}$$

→ So  $score_1 = [0.575, 0.198]$

#### ◆ Step 5.5: Apply Softmax

Softmax( $[0.297, 0.1025]$ ):

Shifted:  $[0.297 - 0.297, 0.1025 - 0.297] = [0, -0.1945]$

makefile

$$\begin{aligned} \text{exp} &= [1.0, e^{-0.1945} \approx 0.8234] \\ \text{sum} &= 1.8234 \end{aligned}$$

→  $[0.5485, 0.4515]$

Softmax( $[0.575, 0.198]$ ):

Shifted:  $[0, -0.377]$

ini

$$\begin{aligned} \text{exp} &= [1.0, e^{-0.377} \approx 0.686] \\ \text{sum} &= 1.686 \rightarrow [0.593, 0.407] \end{aligned}$$

#### ◆ Step 5.6: Compute V (encoder output $\times W_V$ )

$z_1 = [1.15, 1.09]$ :

$$\begin{aligned} V_1[0] &= 1.15 \times 0.4 + 1.09 \times 0.6 = 0.46 + 0.654 = 1.114 \\ V_1[1] &= 1.15 \times 0.5 + 1.09 \times 0.7 = 0.575 + 0.763 = 1.338 \end{aligned}$$

$z_2 = [0.35, 0.41]$ :

$$\begin{aligned} V_2[0] &= 0.35 \times 0.4 + 0.41 \times 0.6 = 0.14 + 0.246 = 0.386 \\ V_2[1] &= 0.35 \times 0.5 + 0.41 \times 0.7 = 0.175 + 0.287 = 0.462 \end{aligned}$$



### ◆ Step 5.7: Final Outputs = softmax × V

For decoder token 0: weights [0.5485, 0.4515]

Copy Edit

$$\begin{aligned}Z_0[0] &= 0.5485 \times 1.114 + 0.4515 \times 0.386 \approx 0.610 + 0.174 = 0.784 \\Z_0[1] &= 0.5485 \times 1.338 + 0.4515 \times 0.462 \approx 0.734 + 0.209 = 0.943\end{aligned}$$

For decoder token 1: weights [0.593, 0.407]

Copy Edit

$$\begin{aligned}Z_1[0] &= 0.593 \times 1.114 + 0.407 \times 0.386 = 0.661 + 0.157 = 0.818 \\Z_1[1] &= 0.593 \times 1.338 + 0.407 \times 0.462 = 0.793 + 0.188 = 0.981\end{aligned}$$

### ✓ Final Cross Attention Outputs:

| Decoder Token | Cross-Attn Output |
|---------------|-------------------|
| "I"           | [0.784, 0.943]    |
| "have"        | [0.818, 0.981]    |



## ✓ STEP 6: Feedforward Network (FFN) + Add & Norm + Output

This step happens **after encoder-decoder attention** for each decoder token.

### 🗉 Recap: Input to FFN

These are the **outputs from encoder-decoder attention**:

- For "I" → [0.784, 0.943]
- For "have" → [0.818, 0.981]

Let's denote:

- $h_0 = [0.784, 0.943]$
- $h_1 = [0.818, 0.981]$

### 🏠 FFN Structure

A typical FFN is two linear layers with a ReLU in between:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

Let's define:



#### First Layer: (2D → 3D)

lua

Copy Edit

```
W1 = [[0.1, 0.2, 0.3],
      [0.4, 0.5, 0.6]]
b1 = [0.01, 0.02, 0.03]
```

#### Second Layer: (3D → 2D)

lua

Copy Edit

```
W2 = [[0.7, 0.8],
      [0.5, 0.6],
      [0.3, 0.4]]
b2 = [0.05, 0.06]
```

### ◆ Step 6.1: Apply First Linear Layer

For  $h_0 = [0.784, 0.943]$ :

Copy Edit

```
a0 = h0 × W1 + b1
= [0.784×0.1 + 0.943×0.4 + 0.01,
```



```
0.784×0.2 + 0.943×0.5 + 0.02,
0.784×0.3 + 0.943×0.6 + 0.03]
= [0.0784 + 0.3772 + 0.01 = 0.4656,
   0.1568 + 0.4715 + 0.02 = 0.6483,
   0.2352 + 0.5658 + 0.03 = 0.831]
→ ReLU(a0) = [0.4656, 0.6483, 0.831]
```

Copy Edit

### ◆ Step 6.2: Apply Second Layer to ReLU Output

lua

Copy Edit

```
FFN0 = a0 × W2 + b2
= [0.4656×0.7 + 0.6483×0.5 + 0.831×0.3 + 0.05,
   0.4656×0.8 + 0.6483×0.6 + 0.831×0.4 + 0.06]
→ First output:
= 0.326 + 0.324 + 0.249 + 0.05 = 0.949
→ Second output:
= 0.373 + 0.389 + 0.332 + 0.06 = 1.154
→ FFN0 = [0.949, 1.154]
```



### ◆ Step 6.3: Add & Norm

This is the **residual connection**:

$$\text{Output} = \text{LayerNorm}(h + \text{FFN}(h))$$

Let's compute  $h_0 + \text{FFN}_0 = [0.784 + 0.949, 0.943 + 1.154] = [1.733, 2.097]$

Now apply **LayerNorm**:

Let:

- $\mu = \text{mean} = (1.733 + 2.097) / 2 = 1.915$
- $\sigma = \text{std} = \sqrt{(\Delta^2)/2}$   
where  $\Delta^2 = (1.733 - 1.915)^2 + (2.097 - 1.915)^2 = (-0.182)^2 + (0.182)^2 = 0.0331 + 0.0331 = 0.0662$   
 $\rightarrow \sigma = \sqrt{(0.0662/2)} = \sqrt{0.0331} \approx 0.182$

Normalize:

$$\text{Norm}_i = \frac{x_i - \mu}{\sigma}$$

$\rightarrow (1.733 - 1.915) / 0.182 = -1$   
 $\rightarrow (2.097 - 1.915) / 0.182 = +1$

So final output for token "I" is: [-1, 1]



### 🔄 Repeat for token "have": $h_1 = [0.818, 0.981]$

Linear 1:

```
yaml
0.818×0.1 + 0.981×0.4 + 0.01 = 0.0818 + 0.3924 + 0.01 = 0.4842
0.818×0.2 + 0.981×0.5 + 0.02 = 0.1636 + 0.4905 + 0.02 = 0.6741
0.818×0.3 + 0.981×0.6 + 0.03 = 0.2454 + 0.5886 + 0.03 = 0.864
→ ReLU: [0.4842, 0.6741, 0.864]
```

Linear 2:

```
0.4842×0.7 + 0.6741×0.5 + 0.864×0.3 + 0.05 = 0.339 + 0.337 + 0.259 + 0.05 = 0.985
0.4842×0.8 + 0.6741×0.6 + 0.864×0.4 + 0.06 = 0.387 + 0.404 + 0.346 + 0.06 = 1.197
→ FFN1 = [0.985, 1.197]
```

Add + Norm:

```
sql
h1 + FFN1 = [0.818 + 0.985, 0.981 + 1.197] = [1.803, 2.178]
mean = 1.9905
std = sqrt(((−0.1875)2 + 0.18752)/2) = sqrt(0.0703) ≈ 0.1875
→ Normalize:
```



```
→ Normalize:  
(1.803 - 1.9905)/0.1875 = -1  
(2.178 - 1.9905)/0.1875 = +1  
→ Final output = [-1, 1]
```

Copy Edit

### ✅ Final Output from Decoder Block (after FFN + LayerNorm)

| Token  | Final Output |
|--------|--------------|
| "I"    | [-1, 1]      |
| "have" | [-1, 1]      |

This goes into the **Linear + Softmax** to predict the next word.

Would you like to conclude with the **final prediction step (linear layer + vocab softmax)** and show which word would be picked next?

📄 👍 🗨 🔊 📌 ↺ ↻ ⬆



### ✅ STEP 7: Final Linear + Softmax — Word Prediction

After all attention + FFN + norm, the decoder gives us final vectors for each token.

From the previous step:

| Token  | Final Output Vector |
|--------|---------------------|
| "I"    | [-1, 1]             |
| "have" | [-1, 1]             |

We'll now:

1. Apply a **linear layer** that maps [-1, 1] into **logits** of vocabulary size
2. Apply **softmax** to get probabilities over the vocabulary
3. Pick the word with the **highest probability**

#### 💎 Vocabulary (example size = 3):

Let's assume we have a tiny vocabulary:

| Index | Word    |
|-------|---------|
| 0     | "a"     |
| 1     | ⬇ "dog" |

1 "dog"

2 "banana"

So we'll map from a 2D vector  $\rightarrow$  3D output (vocab size = 3)

#### ◆ Define Output Projection Layer (2D $\rightarrow$ 3D):

```
lua
W_vocab = [[0.1, 0.2, 0.3],
           [0.4, 0.5, 0.6]]
b_vocab = [0.01, 0.02, 0.03]
```

#### ◆ Step 7.1: Compute Logits for "have" $([-1, 1])$

$$\text{logits} = [-1, 1] \cdot W + b$$

Break it down column-wise:



```
logit[0] = -1×0.1 + 1×0.4 + 0.01 = -0.1 + 0.4 + 0.01 = 0.31
logit[1] = -1×0.2 + 1×0.5 + 0.02 = -0.2 + 0.5 + 0.02 = 0.32
logit[2] = -1×0.3 + 1×0.6 + 0.03 = -0.3 + 0.6 + 0.03 = 0.33
→ logits = [0.31, 0.32, 0.33]
```

#### ◆ Step 7.2: Apply Softmax

Softmax of  $[0.31, 0.32, 0.33]$ :

1. Shift: subtract max = 0.33  $\rightarrow [-0.02, -0.01, 0]$
2. Exponentiate:  $[0.9802, 0.9900, 1.0000]$
3. Sum = 2.9702
4. Divide:
  - $P("a") = 0.9802 / 2.9702 \approx 0.33$
  - $P("dog") = 0.9900 / 2.9702 \approx 0.33$
  - $P("banana") = 1.0000 / 2.9702 \approx 0.34$

🎯 Final Prediction:



### 🔴 Final Prediction:

The decoder assigns the **highest probability (0.34)** to:

"banana" 🍌

So, given the decoder inputs "I", "have", the model **predicts the next word** as:

"banana"

### ✅ End-to-End Summary

| Step | What Happened                                                  |
|------|----------------------------------------------------------------|
| 1    | Encoder processed "CAT", "The" into context vectors $z_1, z_2$ |
| 2    | Decoder took "I", "have" and generated internal vectors        |
| 3    | Masked self-attention calculated local context                 |
| 4    | Cross-attention looked back at encoder's $z_1, z_2$            |
| 5    | FFN transformed and normalized the info                        |
| 6    | Final linear + softmax predicted the next word: "banana"       |



```
+-----+
| INPUT SEQUENCE |
| "CAT", "The"   |
+-----+
```

|



```
+-----+
| STEP 1: Encoder |
| Word Embeddings + PE |
| → Final Encoder Inputs |
+-----+
```

|



```
+-----+
| STEP 2: Encoder |
| Output  $z_1, z_2$  |
|  $z_1 = [1.15, 1.09]$  |
|  $z_2 = [0.35, 0.41]$  |
+-----+
```

|



```
+-----+
| STEP 3: Decoder |
| Input: "I", "have" |
| Word Embeddings + PE |
+-----+
```

|



STEP 4: Masked Self-Attn  
Q, K, V from decoder  
Causal Mask applied  
"have" sees "I", "have"  
 $Z_0 = [0.22, 0.319]$   
 $Z_1 = [0.49, 0.59]$

Copy Edit

STEP 5: Cross Attention  
Q ← decoder Z  
K, V ← encoder  $z_1, z_2$   
Look back at source  
"I" →  $[0.784, 0.943]$   
"have" →  $[0.818, 0.981]$

STEP 6: FFN + Norm  
Two-layer MLP + ReLU  
Add & LayerNorm  
Outputs →  $[-1, 1], [-1, 1]$

Add & LayerNorm  
Outputs →  $[-1, 1], [-1, 1]$

Copy Edit

STEP 7: Linear + Softmax  
Map to vocab logits  
→ "banana" has max prob

#### Final Output:

For the decoder input "I", "have", the model predicts next word = "banana" 🍌

Would you like this ASCII diagram embedded into a .txt or .md file for documentation or study purposes?

Copy Like Comment Quote Share Save Upload